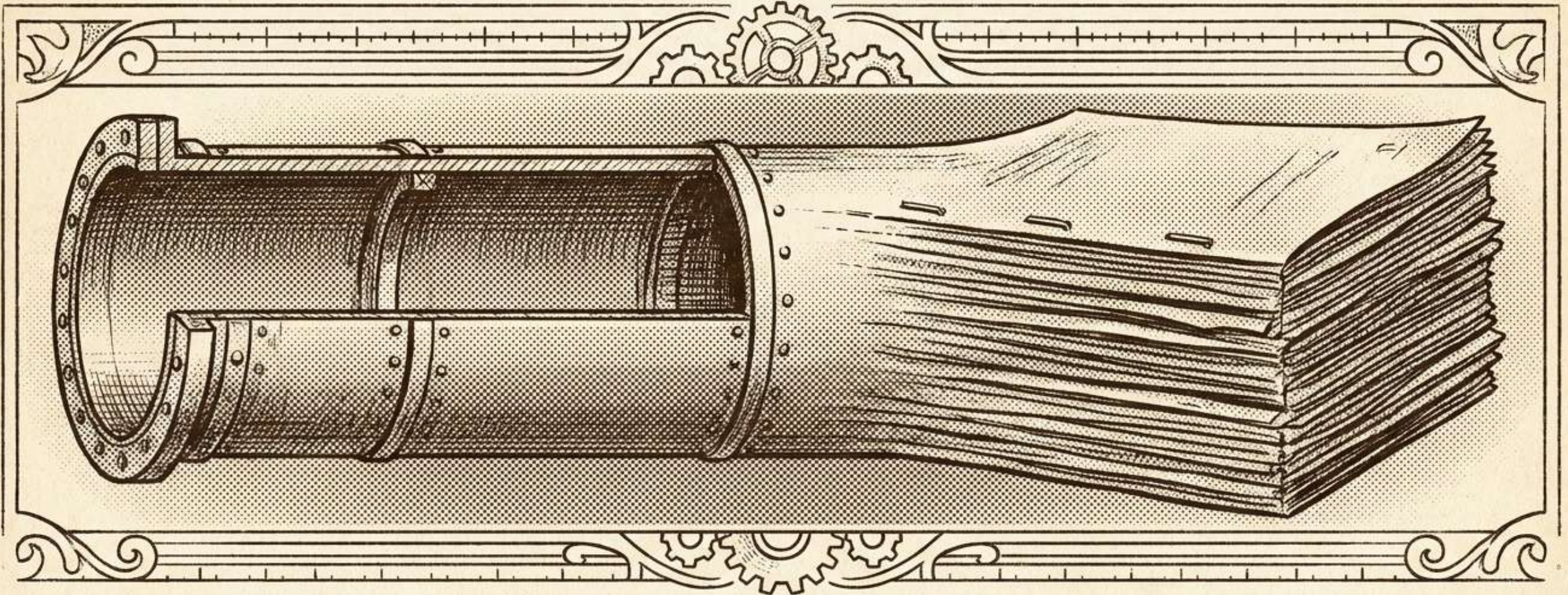


One Does Not Simply Query a Stream

A landscape guide to querying your Kafka data



Before We Start

I'm Viktor Gamov
Developer Advocate at
Confluent
Co-author of Kafka in Action
(Manning)
Java Champion

X / Bluesky: @gamussa
Slides + video:
speaking.gamov.io



Slides + Video

scan for slides, speaker notes, and the recording

`speaking.gamov.io`



I.—THE PROBLEM

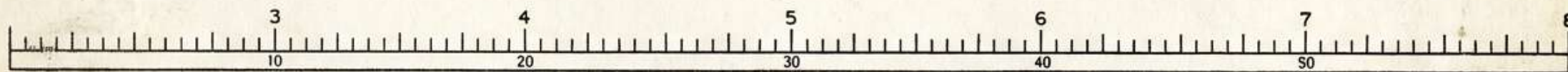


FIG. 3

Simpler Times

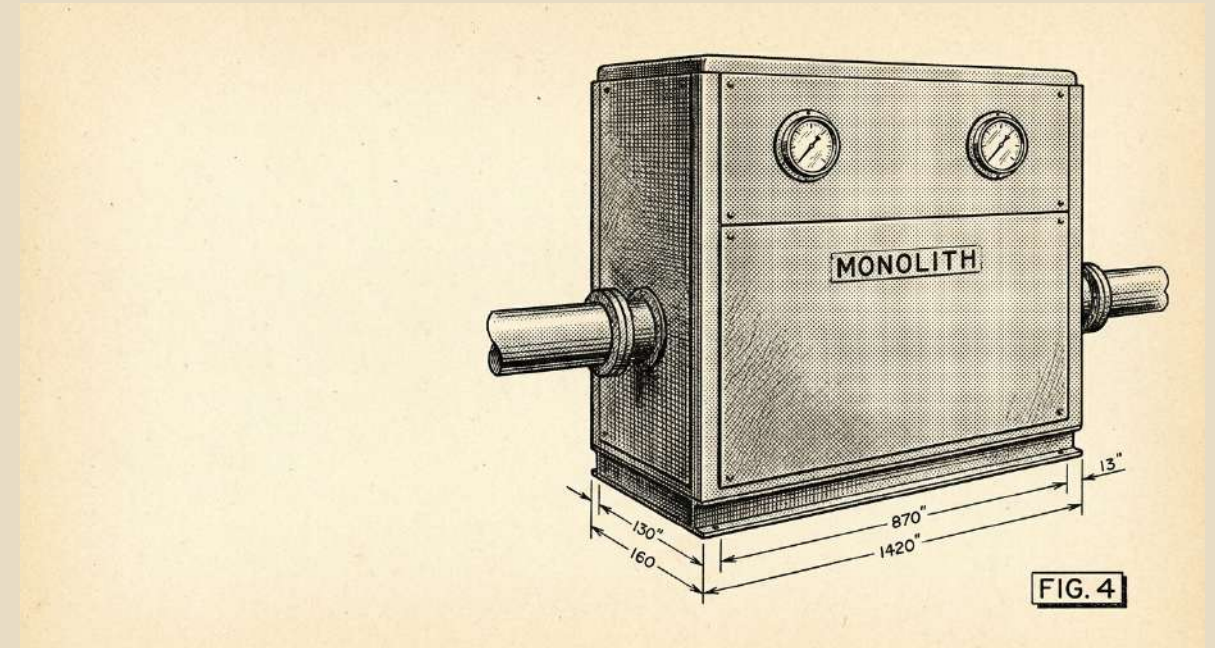
Once upon a time, you had a monolith.

One application. One database.
One SQL query.

```
SELECT * FROM orders  
WHERE status = 'pending';
```

Life was good. You could query anything.

Then someone said "microservices"
– and it
all went sideways.



All Roads Lead to Kafka

Your data is already in Kafka topics.

Events flow through topics in real time.
Data is immutable – once written, it's written.
Kafka is an append-only log, not a database.

So how do you query it?

You can't just `SELECT * FROM kafka_topic`.
(Well... actually... we'll get to that.)

THE LOG

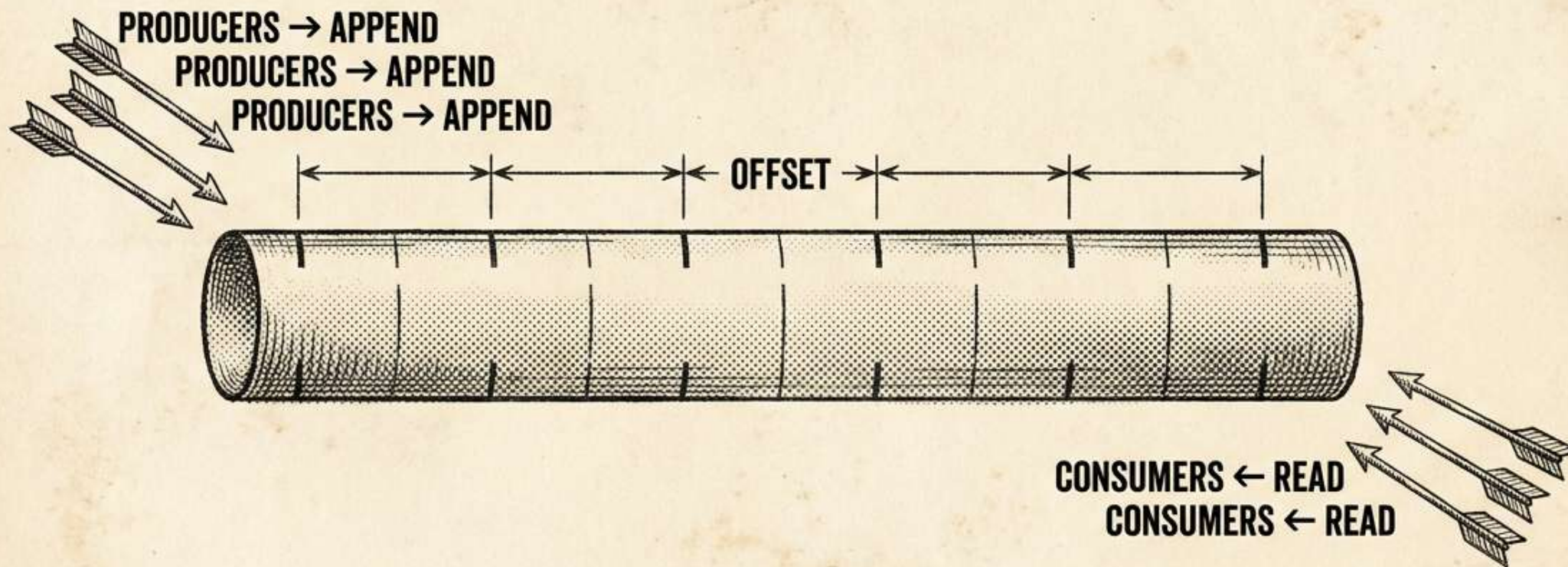


FIG. 5 — THE LOG

REBALANCING — BABY BIRDS



FIG. 8

OLTP vs OLAP

This is the most important slide in this talk.

OLTP – Transactional queries

Get me order #12345

What's the current balance?

Point lookups by key

Low latency, single record

OLAP – Analytical queries

How many orders this hour?

What's the average basket size?

Aggregations across millions

Higher latency, many records

Every solution we'll see optimizes for one of these.

Keep this in your head.

Our Options Tonight

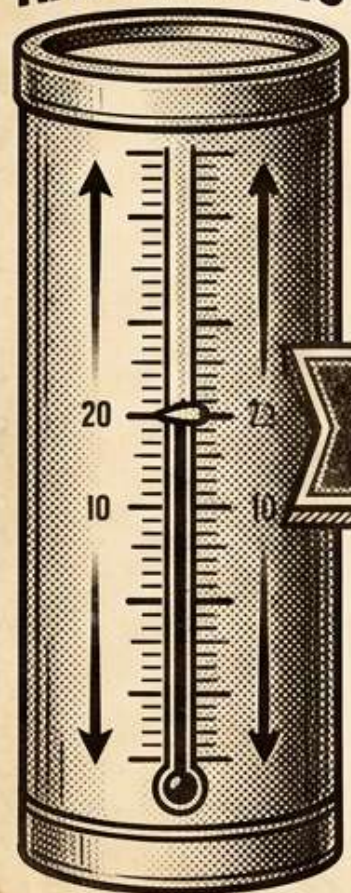
TABLE OF
CONTENTS

1. Kafka Connect + Relational Database
2. Kafka Streams (embedded querying)
3. Streaming SQL databases
4. Real-Time OLAP databases
5. Elasticsearch
6. Cloud Data Warehouses
7. Data Lakes + Table Formats
8. Tableflow (the new kid)

I will not give you a definitive answer.
There are no right solutions. Only trade-offs.

OPTION 1 — CONNECT + RDBMS

KAFKA TOPIC



sink
connector

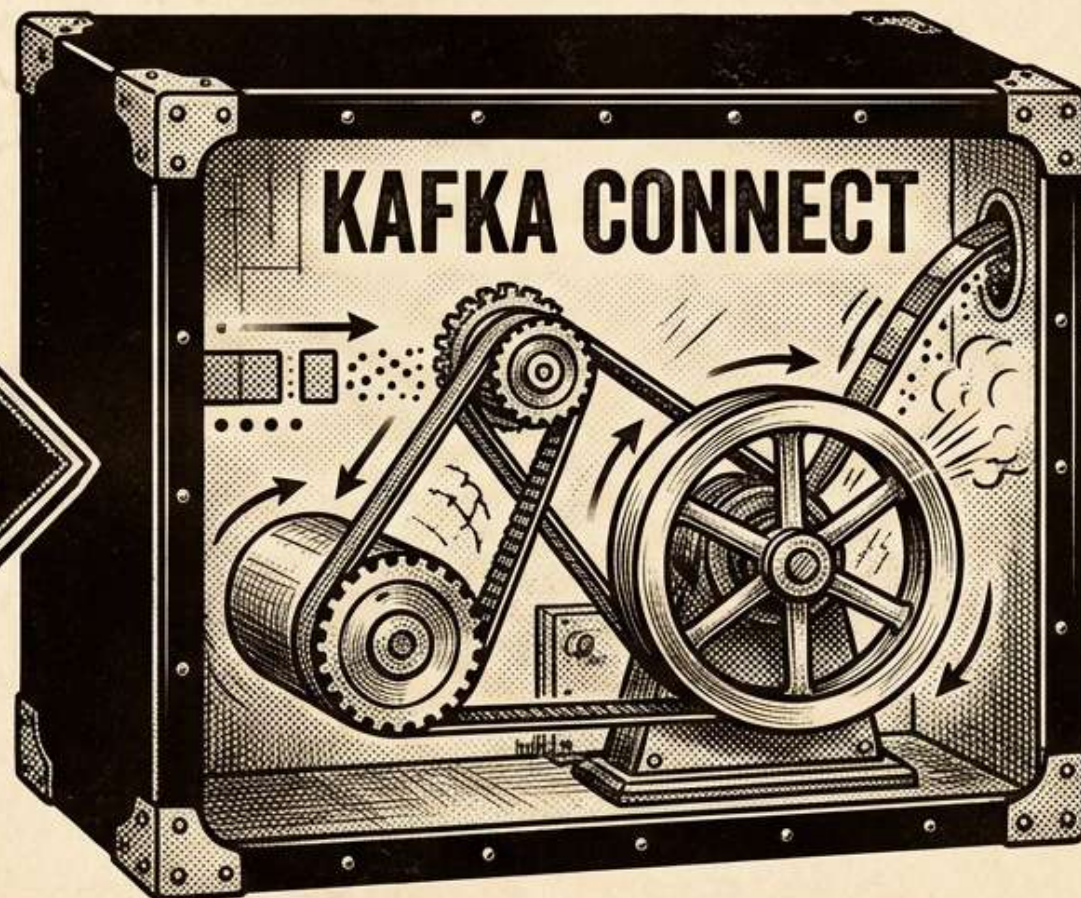


FIG. 6

POSTGRES SQL



JDBC



FIG. 6

your SQL

When Connect + RDBMS Works

REPORT CARD

- + You already know SQL
- + Familiar tooling (pgAdmin, DBeaver, all that jazz)
- + Great for smaller datasets
- + OLTP-friendly – point lookups by key
- Not real-time – there's a lag
- Doesn't scale to millions of events/sec
- You're maintaining another database

If this works for you? It's fine. It's totally fine.

OPTION 2 — KAFKA STREAMS

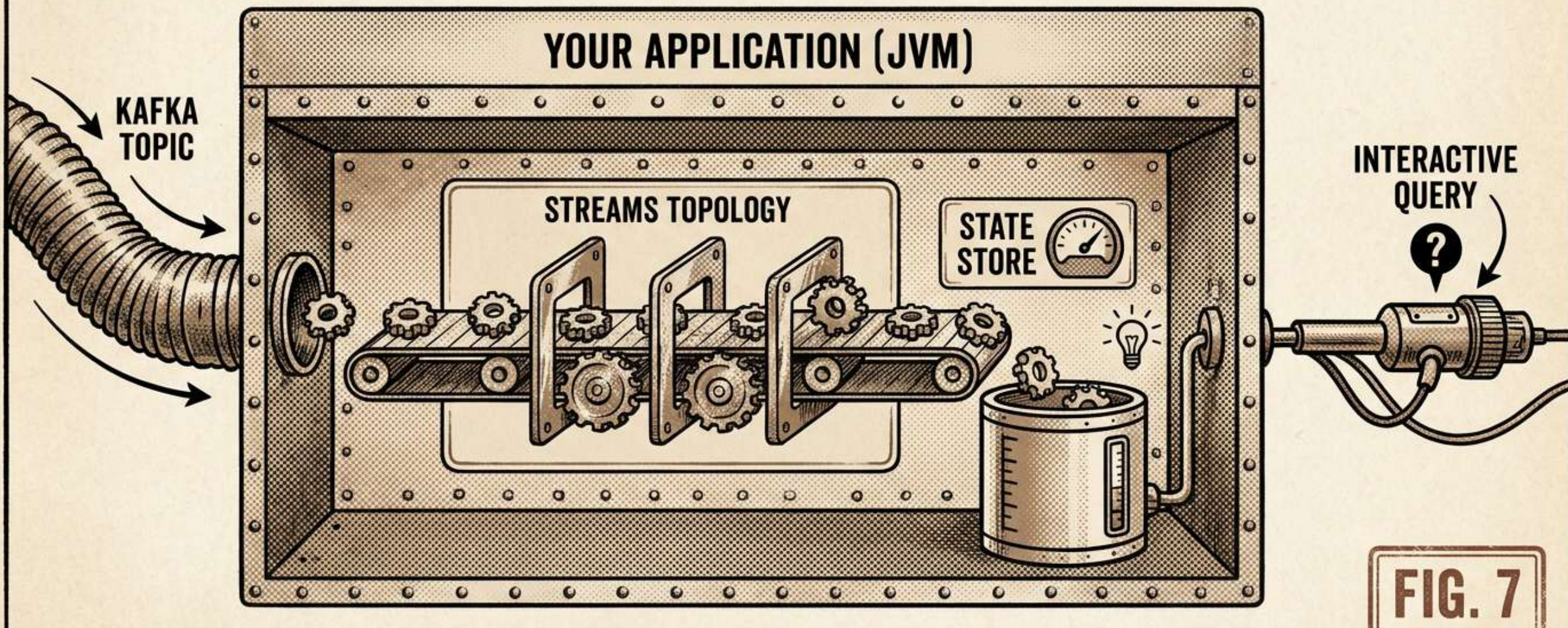


FIG. 7

When Kafka Streams Works

REPORT CARD

- + No external database needed
- + Embedded in your Java/Kotlin app
- + Interactive Queries for OLTP lookups
- + Exactly-once processing
- It's a library, not a service – you manage deployment
- Analytical queries (OLAP) are limited
- Scaling = scaling your app instances

Congratulations, you built your own database.

OPTION 3 — STREAMING SQL

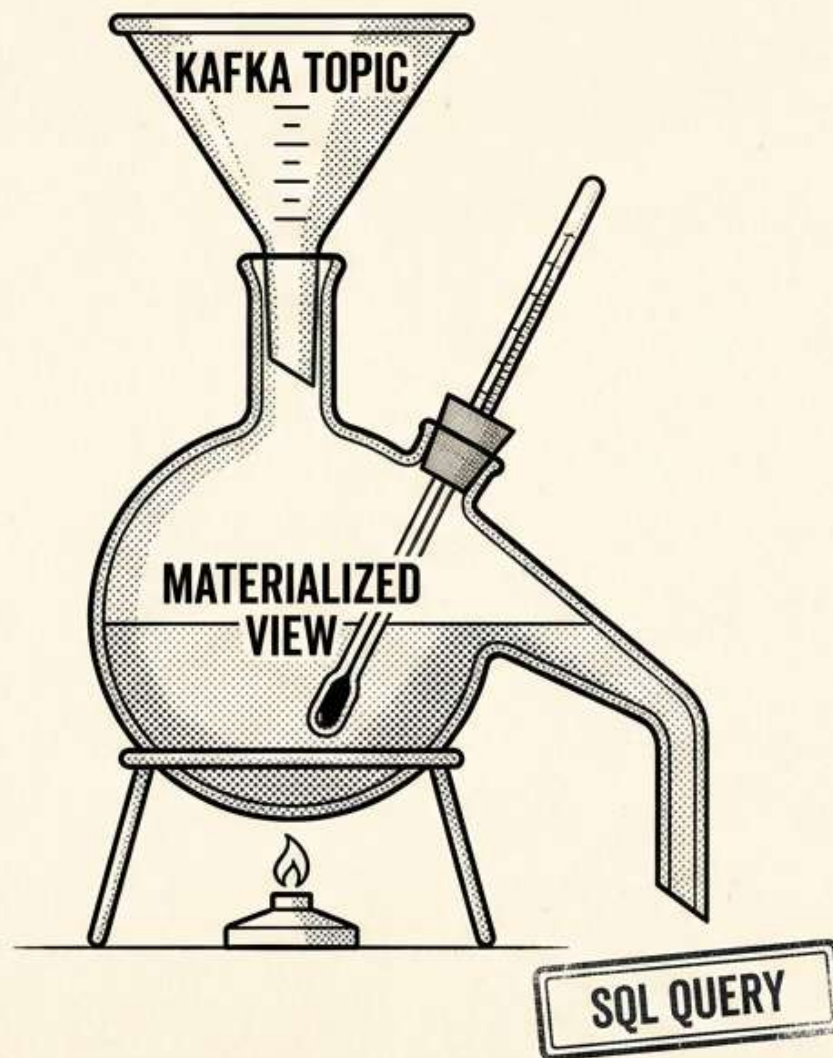


FIG. 9

When Streaming SQL Works

REPORT CARD

- + SQL interface – familiar
- + Continuous materialized views – OLAP on streams
- + No custom code needed
- Another service to deploy and manage
- Scaling characteristics vary wildly between products
- Community support is... developing

Sidebar: Flink has SQL too – but Flink is a processing framework, not a database. Different tool, different job.

OPTION 4 — REAL-TIME OLAP

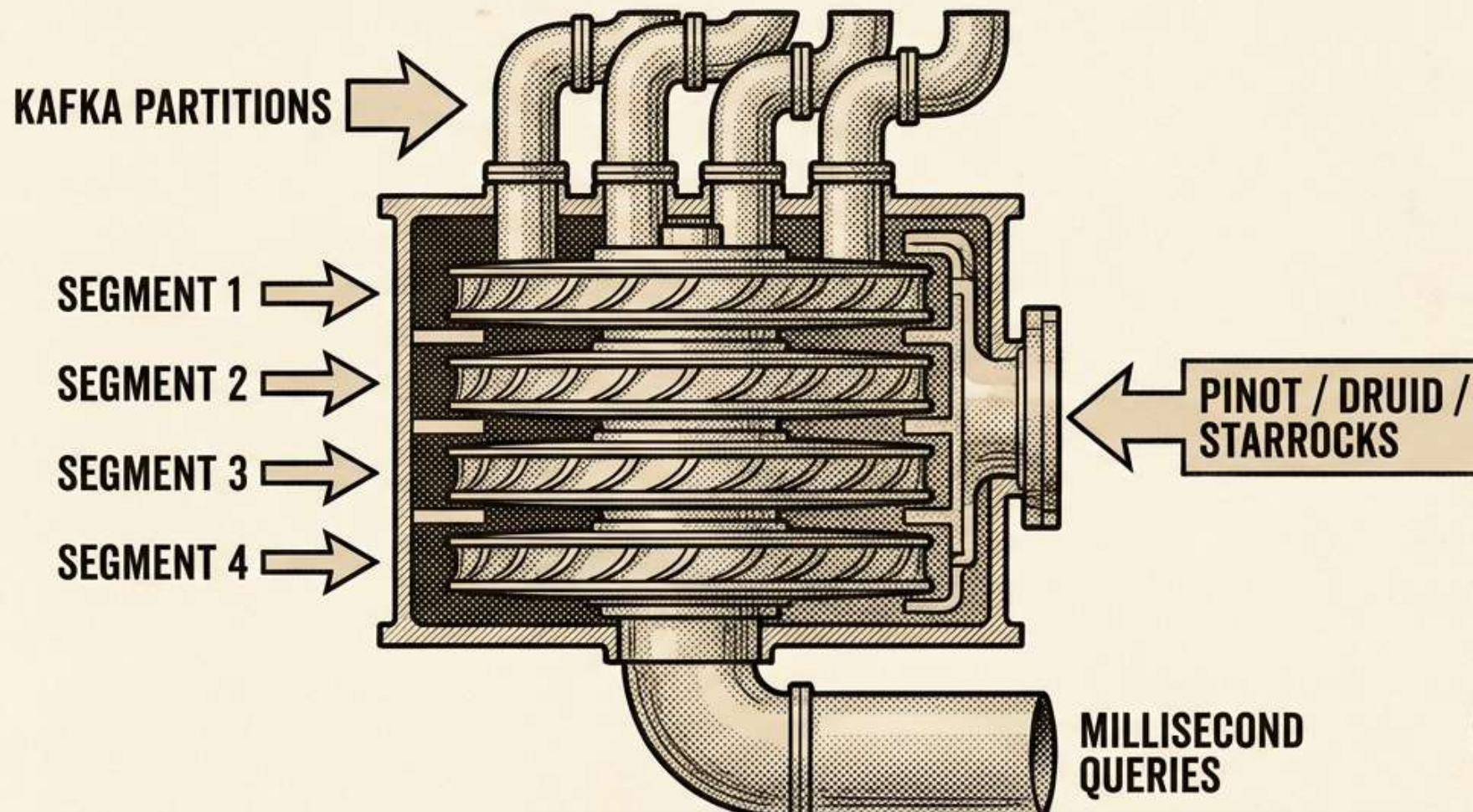


FIG. 10

When Real-Time OLAP Works

REPORT CARD

- + Millisecond query latency at massive scale
- + Built for concurrent analytical queries
- + Kafka is a first-class data source
- Specialized – dedicated OLAP cluster
- Complex operational overhead
- Schema management can be... interesting

OPTION 5 — ELASTICSEARCH

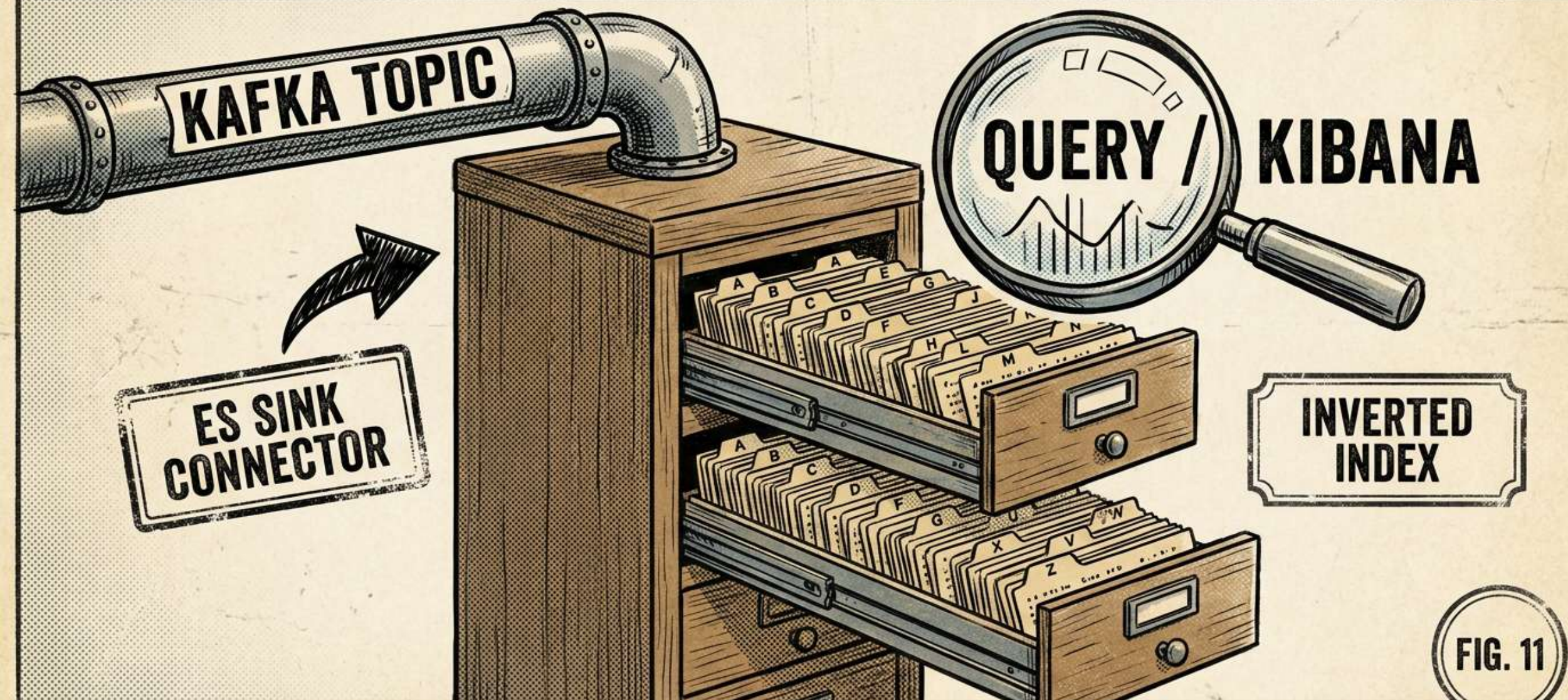


FIG. 11

When Elasticsearch Works

REPORT CARD

- + Full-text search – something none of the others do well
- + Kibana for dashboards out of the box
- + Kafka Connect sink is battle-tested
- + Hybrid: point lookups (OLTP-ish) + aggregations (OLAP-ish)
- Not a streaming engine – destination, not processor
- Schema mapping can get tricky with nested Avro/JSON
- Cluster sizing and shard management at scale

But you all know this already. That's why you're here tonight.

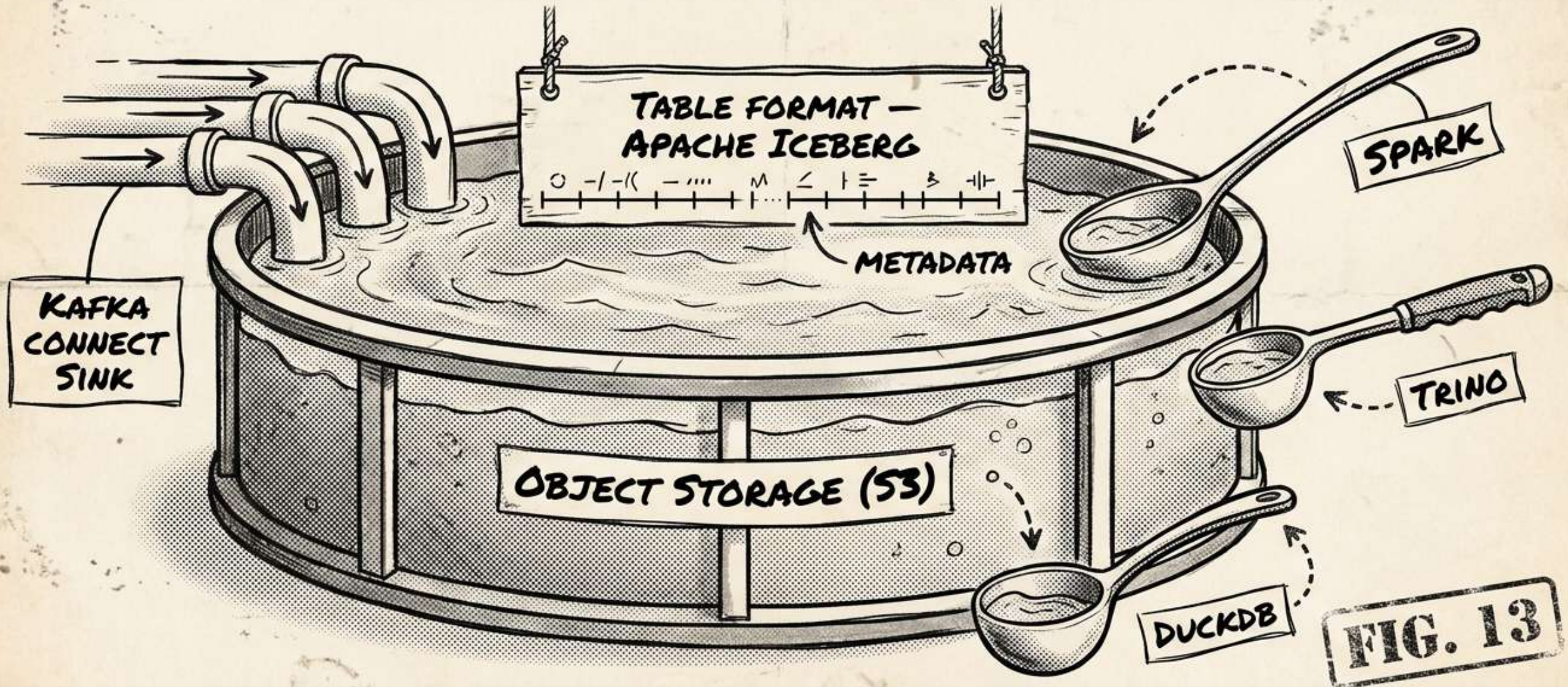
OPTION 6 — CLOUD DATA WAREHOUSES

FIG. 12

- + Massive scale, managed service
- + SQL interface everyone knows
- + Kafka connectors available
- Batch-oriented – even "streaming" modes have latency
- Expensive at high volume
- Structured data bias – semi-structured gets messy

Good for analytics. Not great for real-time.

OPTION 7 — DATA LAKES



OPTION 8 — TABLEFLOW

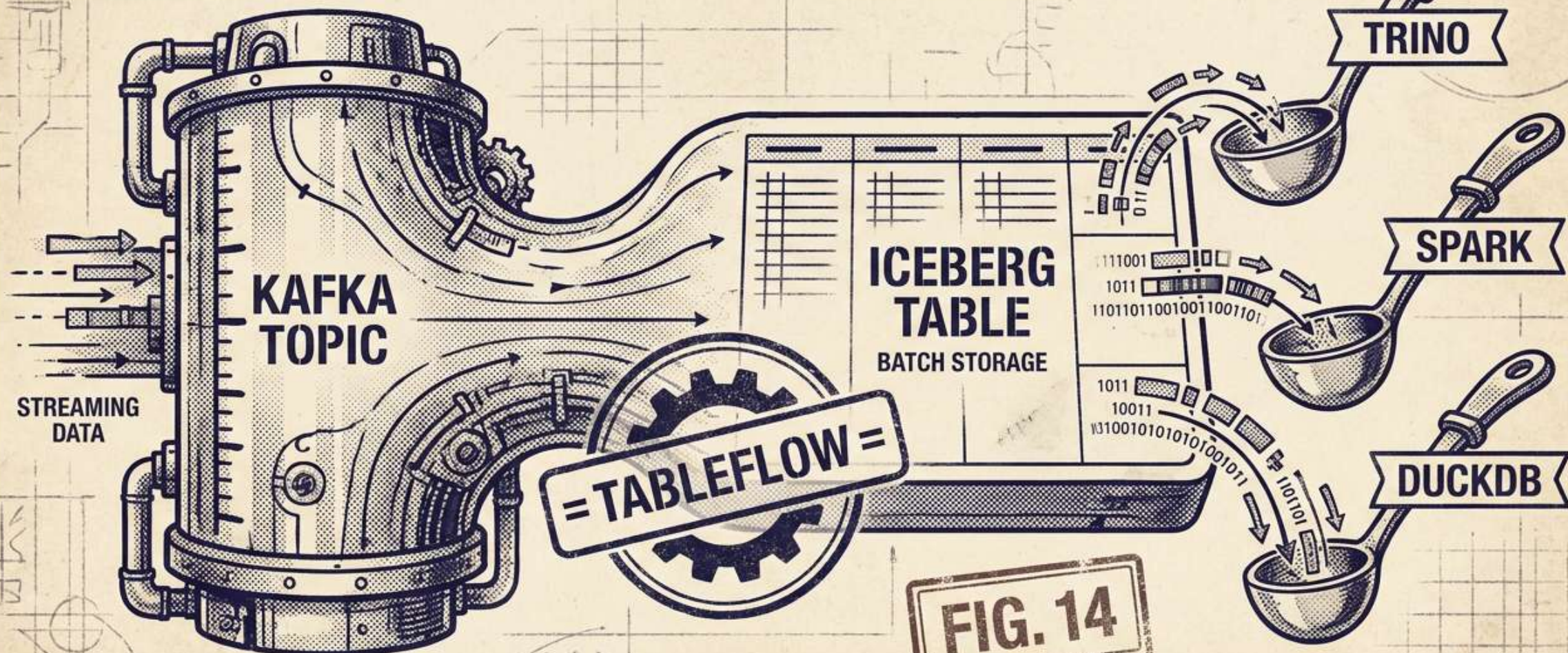
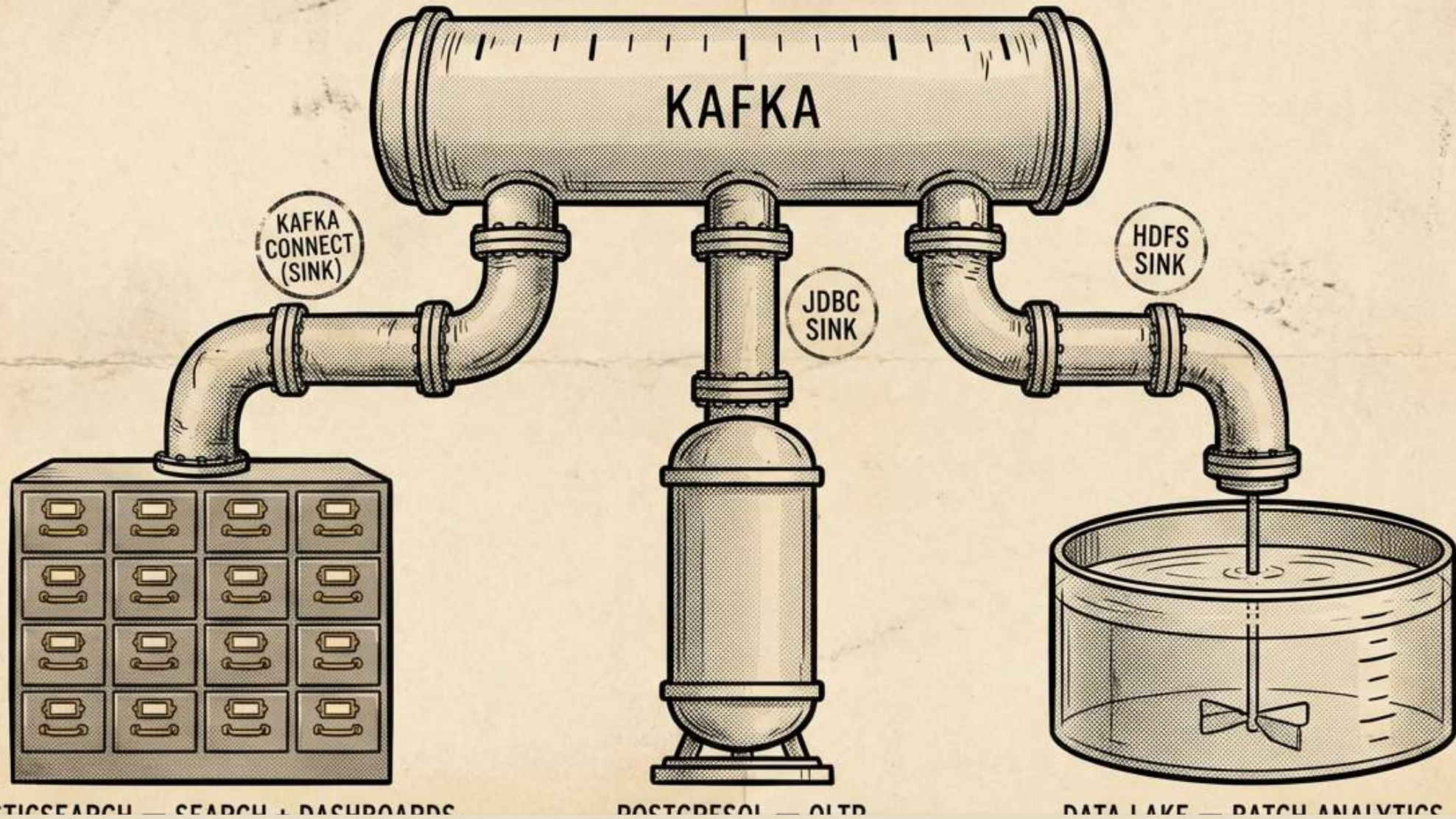


FIG. 14

PRODUCTION SYSTEMS — MULTIPLE VIEWS



★
FIG. 15

III. DECISION FRAMEWORK

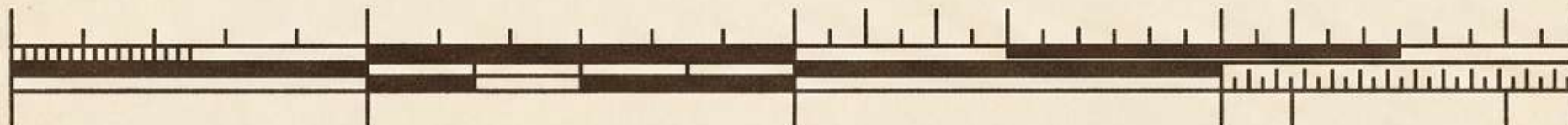


FIG. 16

DECISION KEY

OLTP (point lookups) → Kafka Streams
Connect + RDBMS

OLAP (analytics) → Real-Time OLAP
Streaming SQL

Search + hybrid -> Elasticsearch

```
Batch analytics      -> Data Lake
                    Cloud DWH
                    Tableflow
```

No ideal solutions. Only trade-offs.

Three Things to Consider

THREE
CONSIDERATIONS

1. Familiarity
Sometimes you go with what you know.
That's **NOT** a bad thing.
2. Performance
If consumer lag keeps you up at night,
look at Pinot **or** StarRocks.
3. Community
When you're choosing, think about **where**
you can go ask questions.

The Community Thing

I picked Pascal to learn — not because it was the best language, but because there was a guy in the neighborhood who could help me.

That's why Kafka won.

Not because it's perfect — because people at meetups like this one eat pizza and help each other figure it out.



What to Try This Week

1. Already using Kafka?
Try Interactive Queries with Kafka Streams.
2. Want SQL on streams?
Spin up RisingWave **or** Materialize locally.
3. Need analytics at scale?
Look at Pinot + Kafka connector.
4. On Confluent Cloud?
Enable Tableflow **on** a topic **and** query with DuckDB.
5. Already using Elasticsearch?
Try the Kafka Connect ES sink.
6. Come talk to me – I'll be around after. Pizza first.

Resources

REFERENCES

Slides + video

speaking.gamov.io

Book

Kafka in Action (Manning)

Confluent dev

developer.confluent.io

Streaming Frontiers

my live-stream series

AS ALWAYS, HAVE A NICE DAY



FIG. 18